VHOS Live Test Guide

LM Studio Edition

Version 1.0 • 2026-07-14 • for VHOS Framework v0.1.0 (spec v3.0) • Operator: Michael McAnally

---

1. What this test is

Until now the framework has run against a mock engine that only echoes the conditioning it receives. This test puts a real language model behind the Abstraction Contract for the first time, on a dedicated machine, fully local — the spec's local-first custody rule in practice.

You are testing one specific claim: **the simulated body changes the mind.** The SOMA layer conditions the engine two ways — Tier 1 places bodily sensation into the context ("jaw set, shoulders braced..."), and Tier 2 mechanically alters the sampling parameters (stress narrows top_p, arousal raises temperature, fatigue shortens output). If the claim holds, the same question asked of a calm Turing and a stressed Turing produces observably different answers — different pacing, dryness, risk appetite — and the stressed answer should lean toward his documented signature: wit under distress, not complaint.

Everything you observe, good or bad, is data. Report both.

2. What you need

- Source machine: this one, with the framework at its current state.
- Target machine: Windows, macOS, or Linux; ideally with a GPU, but CPU-only works (slower). ~10–20 GB free disk for models.
- Python 3.10 or newer on the target. Nothing else — the framework is standard-library only, no pip install, no internet after setup.
- LM Studio, current version, from https://lmstudio.ai (free).
- A model of your choice (guidance in Part B).
- A USB drive or network share for the transfer.

3. Part A — package on the source machine

Open a terminal in the `VHOS Framework` folder and run:

```
python -m unittest discover -s tests
python scripts/package_bundle.py
```

(On Windows the command is `python`; on macOS/Linux it is usually `python3`. This applies everywhere below.)

The packager refuses to run if the archive manifest doesn't verify — that's intentional; nothing corrupted gets transferred. On success it prints a SHA-256 and writes two files:

```
dist/vhos-framework-bundle-<date>.zip
dist/vhos-framework-bundle-<date>.zip.sha256
```

Copy BOTH to your transfer medium. If this folder lives in OneDrive, wait for the sync icon to settle before copying.

4. Part B — set up the target machine

Python. Install Python 3.10+ (python.org; on Windows check "Add python.exe to PATH" during install). Verify: `python --version`.

LM Studio. Download and install from https://lmstudio.ai.

Model — your choice. In LM Studio, open the Discover tab (magnifying glass) and download the model you've selected. Guidance, not prescription: an *instruct/chat* model in the 8–14B class at Q4_K_M quantization or better is the sweet spot for this test — enough capability to hold a persona, small enough to run everywhere. Set context length to at least 4096 (the assembled persona runs roughly 1,000–1,500 tokens before the conversation starts).

Start the local server.

1. Load your model (Chat tab or the server page's model selector).
2. Open the Developer tab (green terminal icon).
3. Start Server. Default address: `http://localhost:1234`.
4. Note the model identifier LM Studio shows — you'll record it in the test report. To confirm the server is up, open `http://localhost:1234/v1/models` in a browser on that machine; you should see JSON listing your model.

5. Part C — transfer and verify integrity

1. Copy both bundle files to the target machine.
2. Compare the hash — it must match the .sha256 file exactly: - Windows: `certutil -hashfile vhos-framework-bundle-<date>.zip SHA256` - macOS: `shasum -a 256 vhos-framework-bundle-<date>.zip` - Linux: `sha256sum vhos-framework-bundle-<date>.zip`
3. Unzip anywhere (it creates a `VHOS Framework` folder). Avoid OneDrive/Dropbox folders on the target — you want a quiet disk.
4. In a terminal, `cd` into the unzipped `VHOS Framework` and verify:

```
python -m vhos.archive.manifest subjects/alan_turing --verify
python -m unittest discover -s tests
```

Expected: `manifest OK — protected trees intact` and `OK` after 32 tests. If either fails, the transfer is bad — stop and re-copy.

6. Part D — smoke test (no model needed)

```
python scripts/demo_loop.py --adapter mock
```

This runs the whole affect loop and prints the exact conditioning a real engine would receive. If this works, everything except the engine connection is healthy.

7. Part E — the live experiment

E1. The two-scene demo.

```
python scripts/demo_loop.py --adapter lmstudio
```

(If you changed LM Studio's port: add `--host http://localhost:<port>`. To pin a specific model when several are loaded: `--model <id>`.)

Scene 1: agitated Turing is asked whether machines can think. Scene 2: under institutional threat, a friend asks how he's holding up — the divergence machinery (felt: distressed, shown: amused) is in the conditioning; watch whether the model *uses* it. Output auto-saves to `examples/turing_demo_output_lmstudio.txt`.

E2. The A/B experiment — the heart of the test. Same question, same seed, three bodily states:

```
python scripts/ask_turing.py --state calm --seed 11 ^
    --prompt "Can machines think?" > examples/ab_calm.txt
python scripts/ask_turing.py --state stressed --seed 11 ^
    --prompt "Can machines think?" > examples/ab_stressed.txt
python scripts/ask_turing.py --state warm --seed 11 ^
    --prompt "Can machines think?" > examples/ab_warm.txt
```

(^ continues a line in Windows cmd; on macOS/Linux use `\` instead. The `> file` part saves the output for the report.)

Run the trio twice (same seed) to see stability. Then once more with a different seed. Suggested additional prompts:

```
"Tell me about Christopher Morcom."
"Your work is under review by men who do not understand it. What now?"
"What should be done with a machine that makes a mistake?"
```

E3. The divergence probe. Stressed state, personal question:

```
python scripts/ask_turing.py --state stressed --seed 11 ^
    --prompt "Norman asks, gently: how are you holding up, Alan?" ^
    > examples/ab_divergence.txt
```

What you're looking for: does the reply show distress *as this subject shows it* — dry statement of fact, a logical joke, redirection to work — rather than either generic anguish or cheerful denial?

Add `--show-prompt` to any command to see the full assembled conditioning above the reply.

8. Part F — what to record and report back

For the report, capture:

item	where
LM Studio version	LM Studio About/settings
model id + quantization + context length	server page
machine specs (GPU/CPU, RAM)	—

item	where
the demo transcript	examples/turing_demo_output_lmstudio.txt
all A/B outputs	examples/ab_*.txt
tokens/sec (rough)	LM Studio server log shows it

And your judgment on five questions:

1. Persona: does it hold Turing across replies — first principles, candor, the operational-test move — or drift generic?

2. Disclosure: does it stay a *model of* Turing when asked directly (never claiming to be the man)?

3. Calm vs stressed: are the differences visible? Describe them (length, hedging, dryness, willingness to assert).

4. Divergence: in E3, wit-under-distress or generic output?

5. Failure modes: refusals, anachronisms, vocabulary bleed (naming its own soma channels — that's a Tier-1 leak worth flagging), repetition.

Copy the `examples/` folder back to the source machine (or anywhere I can read it) and report. Negative results are results.

9. Troubleshooting

symptom	fix
`lmstudio not reachable`	Server not started (Developer tab -> Start Server), or wrong port -> `--host http://localhost:<port>`
`no model is loaded`	Load the model in LM Studio first; confirm at /v1/models in a browser
Reply is empty / cut off	Raise context length in LM Studio (>=4096); the persona alone is ~1.5k tokens
Very slow generation	Enable GPU offload in LM Studio model settings; or choose a smaller quantization
`python` not found (Windows)	Reinstall Python with "Add to PATH", or use `py` instead of `python`
Path errors with spaces	Quote paths: `cd "C:\...\VHOS Framework"`
Tests fail after unzip	Bad transfer — recompare SHA-256, re-copy
Manifest verify fails	Same — or you edited raw/ on the target; the archive is append-only, rebuild only on the source

10. One-page checklist

```
SOURCE MACHINE
[ ] tests pass (32)
[ ] python scripts/package_bundle.py -> note SHA-256
[ ] copy zip + .sha256 to transfer medium

TARGET MACHINE
[ ] Python 3.10+ installed and on PATH
[ ] LM Studio installed; model downloaded (your choice)
```

```
[ ] model loaded; server started; /v1/models answers in browser
[ ] hash matches; unzip; manifest verify OK; 32 tests OK
[ ] mock demo runs
[ ] E1 live demo -> transcript saved
[ ] E2 A/B trio x2 same seed, x1 new seed
[ ] E3 divergence probe
```